

ComfoziAI 구매 증빙 인박스

2026 스타트업 영그라운드 MVP 개발 해커톤 · 컴포지 아이템 · 팀 화이팅구리(1인팀)

> 표기 정식 명칭은 **ComfoziAI 구매 증빙 인박스**다. 화면·발표자료·이 문서에서 같은 이름을 쓴다. > 화면 제목 옆의 구매 자료 검토함 은 담당자가 이해하기 쉽게 붙인 **우리말 별칭**이고, 다른 서비스가 아니다. > 회사명은 창업팀 표기를 그대로 따라 **ComfoziAI** 로 붙여 쓴다(정본 요건·회신 문서·창업팀 파일명이 모두 그렇다).

흠어진 구매 증빙을 항목별로 구조화하고, 사람이 확인해야 할 문제를 근거와 함께 표시한 뒤, 사람이 승인한 항목만 ComfoziAI가 받을 수 있는 JSON·CSV로 내보내는 검수 도구다.

> 컴퓨터가 스스로 합치거나 확인 완료로 처리하지 않는다. 판단은 사람이 한다.

구동 URL

 <https://hwaiting-guri.vercel.app/>

로그인이 없다. 주소만 열면 전 기능을 쓸 수 있고, 파일이 없어도 예시 데이터로 바로 보기로 예외 4종과 내보내기까지 확인할 수 있다. 계정·비밀번호가 필요하지 않다.

제공 20건으로 확인하려면 42_해커톤_업로드용_증빙 20건_2026-08-04.csv 를 파일 선택으로 올리면 된다. 확인 대상 4건이 사유와 함께 표시된다.

이 문서의 각 절이 요건 세부설명에 어떻게 대응하는지는 아래와 같다.

요건 세부설명	이 문서에서
구동 URL 또는 로컬 실행 가능	위 링크 · 실행 방법 · 파일널데이 대비 - 배포가 안 열릴 때
소스코드 전체와 실행 방법	실행 방법
지원 범위	지원 범위 → 지원한다
미지원	지원 범위 → 지원하지 않는다 · 범위 밖(OUT of scope)
실패 시도	알려진 제약과 실패한 시도
알려진 오류 기록	개발 중 찾아 고친 오류
비밀값 · 실데이터 사용 금지	사용하지 않은 것 · 환경변수 - 없다
README에 필드 설명과 예시 포함 (요건 ⑦)	출력 스키마
실패 사례와 미지원 형식 기록 (추가 요건)	PDF 공문에서 읽기 → 안 되는 것
(요건 밖) 무엇을 어디까지 검증했는가	검증 범위

실행 방법

```
npm install
npm run dev # http://localhost:3000
```

개발·검증에 사용한 환경은 Node v24.14.1 · npm 11.11.0 이다. package.json 에 engines 를 지정하지 않았으므로 다른 버전에서의 동작은 확인하지 않았다. 그 밖에 따로 설치할 것은 없다.

```
npm run build # 프로덕션 빌드
npx tsc --noEmit # 타입체크
npx eslint . # 린트
npx tsx scripts/verify-20.ts # 자체 테스트 (13개 섹션)
```

자체 테스트는 저장소 상위 폴더의 42_해커톤_업로드용_증빙20건_2026-08-04.csv 를 읽는다. 창업팀 제공 자료라 저장소에 포함하지 않았으므로, 이 파일을 저장소와 같은 상위 폴더에 두어야 테스트가 돈다. 우리가 만든 검사용 파일 3종은 docs/ 에 함께 넣어 두었다.

파일	검사 대상
docs/형식오류_테스트.csv	정수·날짜로 읽을 수 없는 값 10케이스
docs/단위검증_테스트.csv	표준 집합 밖의 단위 10케이스
docs/필수컬럼누락_테스트.csv	9개 필수 컬럼 중 3개만 있는 파일. 매핑 단계에서 걸러 어느 컬럼이 없는지 알려주는지

UTF-8 BOM이 붙어 있어 utf-8-sig 기준으로 처리한다. BOM을 제거하지 않으면 첫 컬럼명이 문서ID가 되어 컬럼 조회가 전부 실패한다.

파이널데이 대비 — 배포가 안 열릴 때 로컬로 띄우는 방법

현장에서 배포 URL이 안 열리거나 인터넷이 없을 수 있다. 로컬 실행이 백업이다. 서버·DB·외부 API를 쓰지 않으므로 노트북 한 대로 전 기능이 돈다.

전날 미리 해 둘 것 — npm install 만 네트워크가 필요하다.

```
npm install # 반드시 전날 미리. node_modules가 있어야 한다
npm run build # 프로덕션 빌드도 미리 만들어 둔다
```

현장에서

```
npx next start # http://localhost:3000
```

npm run dev 가 아니라 next start 를 쓴다. 빌드된 결과를 그대로 띄우는 쪽이 빠르고, 검증도 이 방식으로 했다.

한 가지 사전 준비가 더 있다. PDF 읽기 기능은 pdf.js를 처음 누를 때 내려받는다 (쓰지 않는 사람에게 무거운 파일을 받게 하지 않으려는 선택이다). 그래서 발표 중에 네트워크가 끊기면, 그때까지 PDF를 한 번도 누르지 않았다면 그 순간 실패할 수 있다.

> 발표 직전 한 번 눌러 둔다. PDF 열기 로 공문 하나를 읽어 보면 그 파일이 받아지고, > 이후에는 네트워크와 무관하게 동작한다.

실측 결과다. 화면을 띄운 뒤 네트워크를 끊고 전 기능을 다시 밟았다.

기능	네트워크 끊은 뒤
CSV 20건 업로드·판정	됨 — 확인 대상 4건 · 정상 16건
실질 단가	됨 — kg당 8,600 → 9,556 +11.1%
사람 검수·승인	됨
JSON 내보내기	됨 — 1건 · 18개 필드
PDF 읽기 (한 번도 안 누른 상태)	실패 — pdfjs를 그때 받으려 간다

로컬로 띄우면 이 문제는 없다. 파일이 노트북 안에 있기 때문이다. 위 표는 **배포 URL을 쓰다가 와이파이 가 끊기는 경우**를 재현한 것이다.

제공 20건 CSV는 저장소에 없으니 **USB나 로컬 폴더에 따로 챙겨 간다**. 파일이 없어도 예시 데이터로 바로 보기로 예외 4종과 내보내기까지 보여줄 수 있다.

환경변수 — 없다

.env 파일이 필요 없고 .env.example도 두지 않았다. **읽어야 할 키가 하나도 없기 때문이다**. 판정은 전부 규칙 기반이고 서버에 상태를 두지 않으므로, 인계 시 옮겨야 할 비밀값이 없다.

외부 서비스 의존성 — 없다

구분	사용 여부
외부 API (AI·OCR 포함)	없음 — 판정 4종과 형식 검증 전부 규칙 기반
데이터베이스	없음 — 상태는 브라우저 안에만 있고 localStorage로 백업한다
인증·회원	없음 — 로그인 없이 URL 접속만으로 전체 기능을 쓴다
유료 서비스	없음 — 발생하는 비용이 없다
런타임 의존성	next · react · react-dom · pdfjs-dist 네 개. pdfjs-dist는 PDF를 브라우저 안에서 읽으므로 파일이 밖으로 나가지 않는다
개발 의존성	typescript · @types/* · eslint · eslint-config-next · tailwindcss · @tailwindcss/postcss · tsx. 전부 빌드·검사용이라 실행본에 포함되지 않는다

서버 API 라우트가 없어 전 페이지가 정적으로 생성된다(o (Static) prerendered). 파일을 읽고 판정하는 일이 전부 브라우저 안에서 일어나므로, **중빙 데이터가 외부로 나가지 않는다**.

말로만 두지 않고 배포본에서 실제로 확인했다(2026-08-14 재측정). CSV 20건 업로드 → PDF 공문에서 3건 읽어 등록 → 값 수정 → 승인 → JSON·CSV 내려받기까지 **전 기능을 사용한 뒤** 브라우저가 주고받은 요청 전량을 집계한 결과다.

요청 25건 — 전부 자기 도메인(hwaiting-guri.vercel.app) GET POST·PUT·PATCH 0건 · 외부 도메인 0건

fetch 0건 · XMLHttpRequest 0건 · sendBeacon 0건 · WebSocket 0건

뒤 네 줄은 페이지 로드 시점에 전송 계열 API를 감싸 두고 호출을 세어 얻었다. 정적 파일 요청과 별개로 **앱이 데이터를 내보내려 시도한 적 자체가 없다**는 뜻이다. 우리 소스에도 `fetch` · `XMLHttpRequest` · `WebSocket` · `navigator.sendBeacon` 이 하나도 없다.

PDF를 읽는 `pdf.js` 작업자도 자기 도메인에서 받는다.

```
GET /_next/static/immutable/media/pdf.worker.28jxijx3_bnpc.mjs
```

외부 CDN을 참조하지 않으므로 PDF 기능을 붙인 뒤에도 이 성질은 그대로다. 구매 증빙은 거래처·단가가 담긴 자료라, 판정을 위해 파일을 서버로 올리지 않는 구조를 택했다.

배포 절차

Vercel에 올려 두었다. <https://hwaiting-guri.vercel.app/>

1. Vercel에서 이 저장소를 임포트한다. 프레임워크는 Next.js로 자동 인식된다.
2. 빌드 설정을 바꾸지 않는다(`npm run build`, 출력 `.next`). 환경변수 입력란은 비워 둔다.
3. `main` 브랜치에 push하면 자동으로 다시 배포된다. 수동 단계가 없다.

다른 플랫폼으로 옮길 때도 특별한 준비가 없다. 정적 산출물이라 `npm run build` 결과를 정적 호스팅에 그대로 올려도 동작하고, 옮기면서 다시 연결할 백엔드나 키가 없다.

처리 흐름

CSV/수기 입력 → 9개 필드 구조화 → 정규화 품목명 후보 → 예외 4종 탐지
→ 상태 결정 → 사람 검수(수정·승인·반려) → 승인분만 JSON·CSV 내보내기

관찰값과 수정값을 분리한다

파싱 결과(`observed`)는 절대 변경하지 않고, 사람이 고치는 값은 `current`에 따로 둔다. 덕분에 어느 값이 원본이고 어느 값이 사람이 고친 값인지 항상 구분되며, 변경 이력(`change_log`)은 두 값의 차이로 남는다.

파일	역할
<code>src/lib/types.ts</code>	도메인 타입, 필수 컬럼·표준 단위·중복 키 상수
<code>src/lib/dictionary.ts</code>	품목명 매핑 사전, 약어·수량꼬리 규칙
<code>src/lib/normalize.ts</code>	정규화 후보 생성 (사전 → 규칙 → 데이터 부족)
<code>src/lib/detect.ts</code>	예외 4종 탐지, 중복 그룹 판정
<code>src/lib/parseCsv.ts</code>	BOM 제거, RFC 4180 파서, 필수 컬럼 검증
<code>src/lib/pipeline.ts</code>	전체 파이프라인, 상태 결정, 승인 가능 판정

파일	역할
src/lib/pdfExtract.ts	PDF 표 구조 복원 (추가 요건)
scripts/verify-20.ts	20건 자체 테스트

판정 규칙

예외	판정 조건	상태
missing_required	필수 9개 필드 중 하나라도 공란·공백	needs_review
spec_mismatch	규격이 <code>^\s*기존\s*(.+?)\s*/\s*변경\s*(.+?)\s*\$</code> 에 매칭	on_hold
unit_mismatch	단위가 표준 집합(PK · BOX · EA · PO) 밖이거나 / · , 로 복수 병기	on_hold
duplicate_suspected	7필드 키 완전 일치, 문서ID 오름차순 그룹의 2번째 이후	on_hold

상태 결정은 다음 순서다.

형식 오류 있음 또는 missing_required 있음	→ needs_review
없고 spec/unit/duplicate 중 하나라도 있음	→ on_hold
아무 문제 없음	→ new (승인 가능)

- 필수값이 비어 있으면 채우기 전까지 승인 버튼이 비활성이다.
- 예외가 남은 채로 승인하려면 검수 메모에 판단 근거를 적어야 한다. 전체팀 공지(2026-08-09) §1의 `"아무 검토 없이 예외 상태를 승인하는 흐름은 허용하지 않습니다"`, `"현재 값을 그대로 수용해 승인하면 exception_flags 와 승인 사유를 함께 보존합니다"`를 따른 것이다. 사유를 보존하려면 사유가 있어야 한다. 값을 고치거나 "중복 아님"으로 되돌려 예외를 해소한 경우에는 요구하지 않는다.
- 승인한 뒤 값이나 근거가 바뀌어 승인 조건을 잃으면 승인이 풀린다. 그 승인은 바뀌기 전 상태에 대한 판단이었기 때문이다. 조용히 풀리지 않도록 변경 이력에 승인 해제 로 남기고, 값을 정상으로 되돌려도 자동으로 다시 승인하지는 않는다. 승인은 사람이 다시 누른다.
- 사람이 "중복 아님"으로 되돌리면 중복 플래그를 상태 계산에서 제외해 정상 흐름으로 복귀시킨다.
- 규격 판정과 단위 판정은 서로 배제하지 않는 독립 검사다. 규격이 `기존 ... / 변경 ...` 패턴이어도 단위가 표준이면 unit_mismatch 는 붙지 않는다(DOC-019가 여기 해당).
- 자동 병합·자동 삭제·자동 승인·자동 환산은 하지 않는다.

DOC-020 판단 근거

DOC-020은 규격(`기존 1kg / 변경 4단`)과 단위(`KG/단`)가 각각 독립 규칙에 해당하여 exception_flags 에 spec_mismatch · unit_mismatch 두 개를 모두 담았다. 창업팀 회신(2026-08-06)으로 확인했으며, 규칙 명세 §6-2 표는 대표 유형 하나만 기재된 것이었다. review_status 는 on_hold .

실질 단가 — 규격이 줄었을 때 kg당으로 환산한다

규칙 명세 §4-3이 DOC-019를 이렇게 설명한다.

> 단가는 86,000원 → 86,000원으로 그대로인데 규격이 10kg → 9kg으로 줄었습니다. > **kg당 단가로는 약 11% 실질 인상입니다.** 가격이 안 올랐다고 자동 승인되면 인상을 놓칩니다.

이 계산을 담당자가 머리로 하고 있었다. 화면에서 대신 한다.

문서	규격	단가	화면에 뜨는 것
DOC-019	기 존 10kg / 변경 9kg	86,000 → 86,000	kg당 8,600 → 9,556 +11.1%
SMP-004	기 존 1kg / 변경 900g	43,800 → 43,800	kg당 43,800 → 48,667 +11.1%
DOC-020	기 존 1kg / 변경 4단	28,000 → 22,000	기준이 바뀌어 비교 불가. 공급사 확인 요청

지킨 세 가지

- 1. 산출값이고 판정이 아니다. exception_flags 4종에 손대지 않는다. 공지가 *"예외 4종
- 탐지 규칙과 제공 20건 정답을 변경하지 않습니다"*라고 명시했고, 다섯 번째 플래그를 만들면 20건 정답 대조가 흔들린다. 자체 테스트 13번 첫 항목이 **20건 전체에 계산을 돌린 뒤 플래그와 상태가 그대로인지** 대조한다.
- 1. 승인 조건에 쓰지 않는다. 실질 인상률이 몇 %든 승인은 사람이 누른다.
- 명세 §4-3의 자동 환산·자동 승인 금지를 어기지 않는다.
- 1. 계산할 수 없으면 숫자를 만들지 않는다. 1단이 몇 kg인지는 공급사만 안다.

환산 기준

같은 물리량이면 표기가 달라도 비교할 수 있다. 1kg → 900g 은 기준이 바뀐 게 아니라 표기만 다르다.

차원	단위
질량	mg g kg
부피	ml cc L

표에 없는 단위가 섞이면 환산하지 않는다(kg → 단 , PK → BOX). 대소문자는 무시한다.

거절하는 경우는 이렇다 — 단가를 정수로 못 읽음(형식 검증 결과를 그대로 게이트로 쓴다), 규격 수량이 0, 음수, 숫자 없는 값, 빈 값, 여러 단위가 묶인 규격(2kg×6PK).

표시할 때는 **공문에 적힌 표기를 그대로 쓴다.** 900g 을 0.9kg 으로 바꿔 적으면 원문을 잃는다. 단위당 단가가 100원 미만이면 소수 둘째자리를 살린다 — 999999kg → 1kg 같은 입력에서 반올림하면 0 이 되어 공짜로 읽힌다.

구현은 src/lib/realUnitPrice.ts 한 파일이다. 판정 로직과 섞지 않았다.

값 형식 검증

예외 4종과 **별도 축**이다. 창업팀 회신이 **"예외 4종 탐지 규칙과 제공 20건 정답을 변경하지 않습니다"**라고 명시했으므로, 5번째 플래그를 만들지 않고 `format_errors` 배열에 따로 담는다. `exception_flags` 는 그대로 4종이다.

검사 대상은 **정수·날짜로 해석해야 하는 3개 필드**다. 공지 2절이 지목한 범위가 그것이다.

나머지 6개는 검사하지 않는다. 명세에 `문서 ID = DOC-###`, `원본유형 = PDF|XLSX|IMAGE`수기 처럼 형식이 적혀 있지만, 이건 제공된 합성 20건의 표기이지 제품 규칙으로 굳힐 근거가 없다. 실제 공급사마다 증빙 번호 체계가 다르고, 여기서 형식을 강제하면 20건 밖의 정상 데이터를 거부하게 된다. 단위는 `unit_mismatch` 가 표준 집합 대조를 따로 맡는다.

필드	통과 조건	통과 예	실패 예
<code>기존단가(원)</code> 변경 <code>단가(원)</code>	천단위 콤마를 댄 뒤 <code>^\d+\$</code>	<code>32000</code> · <code>32,000</code>	<code>삼 만 원</code> · <code>33600원</code> · <code>32000.00</code>
<code>적용일</code>	<code>YYYY-MM-DD</code> 이면서 달력에 실재하는 날	<code>2026-08-01</code>	<code>2026/08/01</code> · <code>2026-13-45</code> · <code>2026-02-30</code>

- **읽기를 중단하지 않는다.** 회신 2절대로 파일 전체를 계속 읽고, 해당 행만 `확인 필요` 로 보내면서 실패 이유를 화면에 남긴다.
- **한 행에서 여러 필드가 실패하면 전부 모은다.** 첫 실패에서 멈추지 않는다.
- **공란은 검사하지 않는다.** `missing_required` 가 이미 잡은 문제를 두 번 표시하지 않기 위해서다.
- **자동 보정을 하지 않는다.** `32.000` 이 3만 2천인지 32인지 값만 봐서는 알 수 없고,

단가를 잘못 읽으면 계약 금액이 틀어진다. 판단은 담당자에게 넘긴다.

- 원 단위 단가라 **소수점과 음수는 받지 않는다.** `32000.00` 은 실패 처리된다.
- `2026-02-30` 처럼 형식은 맞지만 달력에 없는 날은, 파싱 결과를 원래 숫자와 다시

대조해 잡는다.

형식 오류도 승인을 막는다 — 명세를 넘어선 판단

명세와 회신은 형식 오류의 상태(`확인 필요`)와 표시만 지정했고, 승인 차단까지는 지정하지 않았다. 그럼에도 막기로 한 이유는, 단가를 정수로 읽지 못한 채 승인하면 내보내기에서 숫자 필드를 채울 수 없어 **앞단 설계가 받을 수 없는 출력이 나가기** 때문이다. 빈 값과 해석 불가 값은 "쓸 수 없는 값"이라는 점에서 같아서 같은 취급을 했다.

값을 고치면 즉시 풀린다. 영구 차단이 아니다.

정규화 품목명 — 사전을 사용했다

품목명 매핑 사전(19종)을 품목 마스터로 사용한다. 창업팀이 제공한 20건 기준 사전이며, `src/lib/dictionary.ts` 에 JSON 형태로 두고 조회한다.

조회는 3단이다.

1. **사전 조회** — 원문 품목명으로 찾는다

2. **규칙 기반** — 사전에 없으면 약어 전개(냉감튀 → 냉동 감자튀김 , S/O → 소스 , BBQ → 바비큐),
수량·규격 꼬리 제거, 공백 정리를 적용한다

1. **데이터 부족** — 후보를 만들지 못하면 빈칸이 아니라 데이터 부족 으로 표시한다

사전은 **품목명으로 조회**하므로, 금지 대상인 "문서ID별 예외 결과 하드코딩"과는 다르다. 문서ID를 바꿔도
판정이 동일함을 자체 테스트로 확인한다(아래 참조).

자동 정규화의 정확도는 평가 대상이 아니라는 안내에 따라, 후보를 표시하고 사람이 고칠 수 있게 하는
데까지만 구현했다.

출력 스키마

승인된 항목만 내보낸다. 승인 전·보류·반려는 제외된다.

```
{
  "uid": "1:DOC-001",
  "doc_id": "DOC-001",
  "source_type": "PDF",
  "supplier_name": "가온푸드(예시)",
  "raw_item_name": "토마토살사S/O",
  "normalized_item_name": "토마토 살사 소스",
  "spec": "4kg/PK",
  "unit": "PK",
  "price_before": 32000,
  "price_after": 33600,
  "effective_date": "2026-08-01",
  "review_status": "approved",
  "exception_flags": [],
  "format_errors": [],
  "source_ref": {
    "input_method": "file",
    "file_name": "42_해커톤_업로드용_증빙20건_2026-08-04.csv",
    "row_no": 2,
    "page_no": null
  },
  "reviewed_at": "2026-08-10T14:03:00+09:00",
  "review_memo": "",
  "change_log": [
    { "at": "2026-08-10T14:01:00+09:00", "field": "normalized_item_name",
      "from": "토마토살사소스", "to": "토마토 살사 소스", "action": "edit" }
  ]
}
```

필드	설명
uid	행을 유일하게 식별하는 값. 명세 권장안에는 없지만 덧붙였다. 문서ID는 공문이 재발송되거나 파일을 두 번 올리면 중복되므로, 받는 쪽이 이 값을 키로 쓰면 안전하다
doc_id	원본 증빙 식별자. 유일하지 않을 수 있다

필드	설명
source_type	증빙이 원래 들어온 형태(PDF · XLSX · IMAGE · 수기). 사람이 고칠 수 있는 표시용 값이라 출처 근거로는 쓰지 않는다
raw_item_name / normalized_item_name	공급사 표기 원문 / 정규화 후보 또는 사람이 고친 값
price_before / price_after	인상 전·후 단가. 동결·인하도 있다
review_status	new · needs_review · on_hold · approved · rejected
exception_flags	missing_required · spec_mismatch · unit_mismatch · duplicate_suspected 순서로 담는다
format_errors	정수·날짜로 읽지 못한 필드(field · value · reason). 형식 오류가 있으면 승인이 막히므로 출력에서는 항상 빈 배열이다
source_ref.input_method	어떤 경로로 들어왔는지. file · pdf · manual . source_type 과 달리 사람이 고 칠 수 없어 출처를 되짚는 기준이 된다
source_ref.row_no	헤더를 1행으로 세는 1-indexed 행 번호. 파일 인입이 아니면 null
source_ref.page_no	PDF 공문에서 읽었으면 그 표가 있던 쪽 번호. 그 밖에는 null
change_log	사람이 값을 고친 내역. 시각·필드·이전값·수정값·동작

CSV 평탄화 규칙

CSV는 위 구조를 평탄화한다. 열 순서는 아래와 같고 인코딩은 UTF-8이다.

```
uid, doc_id, source_type, supplier_name, raw_item_name, normalized_item_name,
spec, unit, price_before, price_after, effective_date,
review_status, exception_flags, reviewed_at, review_memo,
source_input_method, source_file_name, source_row_no, source_page_no,
change_log_count, change_log_json
```

- 중첩 필드 source_ref.* 는 source_ 접두사를 붙여 편다(file_name → source_file_name).
- exception_flags 는 세미콜론(;)으로 잇는다. 쉼표를 쓰면 열 구분자와 겹쳐 읽기 어려워진다.
- change_log 는 배열이라 열 하나로 펼 수 없어 change_log_count (건수)와

change_log_json (JSON 문자열) 두 열로 남긴다. 건수만 두면 CSV만 받는 쪽이 이력을 잃는다.

- 맨 앞에 UTF-8 BOM을 붙인다. 없으면 엑셀이 UTF-8을 인식하지 못해 한글이 깨진다.
- 값에 쉼표·따옴표·줄바꿈이 있으면 RFC 4180대로 따옴표로 감싸고 내부 따옴표는 두 번 쓴다.
- 줄바꿈은 CRLF .

샘플 파일

실제 화면에서 검수하고 내려받은 결과를 docs/samples/ 에 넣어 두었다. 손으로 쓴 예시가 아니라 앱이 만든 출력 그대로다.

파일	내용
docs/samples/comfozi_approved_sample.json	승인 4건
docs/samples/comfozi_approved_sample.csv	같은 4건을 평탄화한 것

샘플에 일부러 네 가지 경로를 섞었다.

문서	담긴 경로
DOC-001 · DOC-003	예외 없이 바로 승인
DOC-016	값을 고쳐 예외를 해소한 뒤 승인 — 비어 있던 적용일을 채웠고 change_log 에 수정과 승인이 2건 남았다
DOC-019	예외를 수용해 승인 — exception_flags 에 spec_mismatch 가 그대로 있고 review_memo 에 판단 근거가 있다

승인하지 않은 16건과 반려한 DOC-018은 두 파일 모두에 들어 있지 않다.

자체 테스트

```
npx tsx scripts/verify-20.ts
```

열세 가지를 확인한다.

- 1. 20건 판정이 창업팀 기대 결과와 일치하는가 (정정 공지 §4 표 기준)
- 2. 대기 사유 문구가 플래그마다 비어 있지 않은가 — 사유가 비면 화면에 표시할 게 없다는 뜻이라 실패로 본다
- 3. 하드코딩 없이 판정하는가

- (a) 문서ID를 전부 바꾸고 행 순서를 뒤집어도 20건 판정이 동일한지 - (b) 문서ID 대소 순서를 반대로 매기면 중복 그룹의 기준 항목이 따라 바뀌는지

- 1. 같은 파일을 두 번 올리면 1회차 판정은 그대로고 2회차 20건에 중복 의심이 붙는가, 문서ID가 겹쳐도 uid 40개가 유일한가

- 1. 값 형식 검증 — 아래 표 참조

5번은 제공 20건이 전부 정상값이라 드러나지 않는 구멍이라, 깨진 값을 일부러 넣은 자체 제작 파일 docs/형식 오류_테스트.csv (10건)로 검사한다.

문서	넣은 값	기대
FMT-001	기존단가 삼만원	형식 오류 1건 · 승인 차단
FMT-002	변경단가 33600원	형식 오류 1건 · 승인 차단
FMT-003	적용일 2026-13-45	형식 오류 1건 · 승인 차단

문서	넣은 값	기대
FMT-004	적용일 2026-02-30	형식 오류 1건 · 승인 차단 (달력에 없는 날)
FMT-005	적용일 2026/08/01	형식 오류 1건 · 승인 차단
FMT-006	세 필드 동시에 깨짐	형식 오류 3건 — 첫 실패에서 멈추지 않는다
FMT-007	기준단가 32,000	정상 — 천단위 콤마는 읽는다
FMT-008	적용일 공란	형식 오류 0건 + missing_required — 중복 표시하지 않는다
FMT-009	기준단가 32000.00	형식 오류 1건 — 원 단위라 소수점을 받지 않는다
FMT-010	전부 정상	예외 없음 · 승인 가능

여기에 더해 두 가지를 같이 본다.

- 형식 검증이 exception_flags 에 5번째 플래그를 끼워넣지 않았는가
- FMT-001의 단가를 32000 으로 고치면 오류가 풀리고 승인이 열리는가

6·7번은 승인 이후를 본다.

1. 승인한 뒤 값이 깨지면 승인이 풀리고 이력에 unapprove 가 남는가, 값을 되돌려도

자동으로 재승인되지 않는가

1. 예외를 수용해 승인할 때 판단 근거를 요구하는가 — 메모 없이 차단 / 메모 있으면 통과 /

예외가 없으면 요구하지 않음 / "중복 아님"으로 해소한 경우에도 요구하지 않음 / 승인 후 메모를 지우면 승인이 풀림

8번은 20건으로 실행되지 않던 코드 경로를 검사한다. 20건의 단위는 PK · BOX · PO 와 복수 병기 KG/단 뿐
이라, 표준 집합 밖의 단일 단위 분기가 한 번도 실행된 적이 없었다. 2026-08-12 샘플 공문에 봉 · 망 · 말
· 팩 · 캔 · 박스 가 나온 것을 보고 docs/단위검증_테스트.csv (10건)를 만들어 따로 검사한다.

단위	기대
봉 망 말 팩 캔	unit_mismatch
박스	unit_mismatch — BOX 의 한글 표기여도 표준 집합 밖이다
BOX	정상 (대조군)
pk	unit_mismatch — 소문자를 대문자로 보정하지 않는다
" PK "	정상 — 파서가 셀을 읽을 때 앞뒤 공백을 다듬는다
EA/BOX	unit_mismatch — 복수 병기 경로

여기에 더해 단위를 임의로 바꾼 항목이 0건인지 확인한다. 관찰값을 그대로 두고 사람이 판단하게 하는
것이 명세의 자동 환산 금지 원칙이다.

9번은 내보내기를 본다. 20건 중 DOC-001을 그냥 승인, DOC-019를 근거 메모와 함께 수용 승인, DOC-
002를 반려한 상태에서 출력을 만들어 검사한다.

- 승인 2건만 나가고 반려·미승인은 빠지는가

- 예외를 수용해 승인한 DOC-019에 `spec_mismatch` 와 검수 메모가 남는가 (체크리스트 3번)
- 단가가 문자열이 아니라 정수로 나가는가
- CSV 헤더 21열·BOM·행 수가 맞는가, JSON을 다시 파싱할 수 있는가
- 승인 뒤 단가를 깨뜨리면 출력에서 빠지는가 — 승인이 폴리로 깨진 값이 앞단으로 가지 않는다
- 같은 문서ID 2건을 모두 승인해도 `uid` 로 구분되는가, 그리고 이중 반영 경고가 뜨는가
- 메모에 따옴표·쉼표·줄바꿈이 있어도 CSV 왕복 후 원문 그대로인가 (RFC 4180)

10번은 수기 등록이 파일 인입과 같은 경로를 타는지 본다.

- 출처가 `manual` 로 남고 파일명·행 번호가 `null` 인가
- 사전 조화·필수값 검증·형식 검증·단위 판정이 폼 입력에도 그대로 적용되는가
- 파일로 들어온 DOC-001과 같은 내용을 수기로 넣으면 중복 의심이 붙는가 — 이것이

별도 경로를 만들지 않은 이유다

- 수기 항목을 승인하면 내보내기에 `input_method: "manual"` 로 실리는가
- 수기 등록 후에도 기존 20건 판정이 그대로인가

11번은 내장 예시 데이터를 본다.

- 예시 10건에서 예외 4종과 형식 오류가 모두 나오는가
- 문서ID를 바꿔도 판정이 같은가 — 예시 화면도 데이터에서 계산된다는 증거
- 제공 20건의 값이 섞여 들어가지 않았는가(규정 4번)
- 각 항목이 원본 CSV 한 줄을 들고 있는가

12번은 PDF 표 읽기를 본다. 샘플 공문 4종에서 품목을 하나도 빠뜨리지 않는지, 단가에서 콤마와 원 을 떼어 그대로 쓸 수 있는지, 읽은 항목이 같은 파이프라인을 타는지, 깨진 PDF에 이유를 알려 주는지 확인 한다. 창업팀 자료라 저장소에 없으므로, 샘플 공문 폴더가 없으면 이 섹션은 건너뀐다.

출처도 함께 고정한다. PDF로 넣은 항목이 `input_method: "pdf"` 로 남고 파일명·쪽 번호를 들고 있는지, 목록 배지가 PDF 1쪽 · 수기 · 2행 으로 갈리는지, 승인 후 내보낸 CSV에 파일명과 쪽 번호가 실리는지 본다. pdf 를 manual 로 되돌려 보면 이 검사 3개가 실패한다.

13번은 실질 단가를 본다(21항목). 첫 항목이 가장 중요하다 — 20건 전체에 계산을 돌린 뒤 예외 플래그 와 상태가 그대로인지 대조한다. 산출값이 판정을 건드리면 20건 정답 대조가 흔들리기 때문이다.

- DOC-019가 kg당 8,600 → 9,556 , +11.1% 로 나오는가 — 명세 §4-3의 "약 11%"와 같은 값
- DOC-020(1kg → 4단)은 숫자를 만들지 않고 공급사 확인을 안내하는가
- 규격 변경이 없는 항목에는 카드를 만들지 않는가
- 환산 경계 5종 — kg↔g · L↔ml ·대소문자는 환산, kg→단 · PK→BOX 는 거절
- 규격 문자열 경계 8종 — 아주 큰 수량·소수점·콤마 · g↔mg 는 계산,

숫자 없는 값·빈 값·음수는 거절

- 단가를 정수로 못 읽으면, 수량이 0이면, 여러 단위가 묶인 규격이면 멈추는가
- ★ 예시 데이터에서 실질 단가가 실제로 계산되어 보이는가 — 심사위원은 파일 없이

예시 데이터로 들어온다. 여기서 안 보이면 없는 기능과 같다. 이 검사가 없던 동안 실제로 화면이 비어 있었다(개발 중 찾아 고친 오류 08-15 참조)

최근 실행 결과 전문은 docs/검증결과_전체_2026-08-15.pdf 에 있다(13섹션, 6쪽). 콘솔 출력을 다듬지 않고 그대로 옮겼고, 위 네트워크 실측도 함께 담았다.

이전 회차 기록도 남겨 두었다. 무엇을 언제부터 검사하기 시작했는지 따라갈 수 있다.

문서	시점	담긴 범위
docs/검증결과_전체_2026-08-15.pdf	최신	13섹션 전체 (실질 단가 추가)
docs/검증결과_전체_2026-08-14.pdf	2026-08-14	12섹션 (실질 단가 이전)
docs/검증결과_전체_2026-08-13.pdf	2026-08-13	9섹션 (수기 등록·예시 데이터·PDF 이전)
docs/검증결과_형식검증_2026-08-12.pdf	2026-08-12	값 형식 검증 도입 시점
docs/검증결과_20건_2026-08-05.pdf	2026-08-05	20건 정답표 최초 대조

검증 범위 — 무엇을 어디까지 재봤는가

자체 테스트는 Node에서 도는 판정 검사다. 그것만으로는 브라우저에서 일어나는 일과 받은 적 없는 데이터 알 수 없어, 배포본에서 직접 알아 본 범위를 따로 적는다.

무엇을	어떻게	결과
판정 정확도	자체 테스트 13섹션 (<code>npx tsx scripts/verify-20.ts</code>)	전체 통과. 20건 정답표 전건 일치
정적 검증	타입체크 · ESLint · 프로덕션 빌드	종료코드 0
브라우저	Chromium 141 · Firefox 142 · WebKit 26 에서 배포본으로 핵심 흐름	세 엔진 동일. 사파리 결함 1건을 여기서 찾았다
제공 데이터	20건 CSV 업로드	확인 대상 4건 · 정상 16건 (정답표와 같음)
예시 데이터	파일 없이 예시 데이터로 바로 보기 (심사위원 진입 경로)	10건 · 예외 4종 전부 + 형식 오류 1건 · 실질 단가 계산값 표시
받은 적 없는 데이터	직접 만든 CSV 39건 — 소수점·음수·거대 숫자·날짜 형식 3종·이모지·200자 품목명·따옴표·BOM 없는 파일, 규격 문자열 경계 15종	크래시 0. 파싱 실패는 전부 확인 필요로 떨어짐
경계 파일	헤더만 있는 파일 · 컬럼이 빠진 파일 · 1건짜리 파일	빈 목록 안내 / 필수 컬럼이 없습니다 : 적용일 / 정상 처리
부하	1,000건 CSV(제공 데이터의 50배)	3.7초에 처리, 항목 선택 828ms, 멈추지 않음
화면 폭	1280 · 1024 · 820 · 600px	가로 스크롤 0. 제목·표 모두 보임

무엇을	어떻게	결과
키보드만 으로	값 수정 → Tab → Enter로 승인	됨. 마우스 없이 검수 가능
PDF 공문	샘플 4종	6/6 · 4/4 · 3/3 · 1/1, 품목 누락 0
증빙 유출	전 기능을 쓴 뒤 주고받은 요청 전량 확인	25건 전부 자기 도메인 GET. 외 부 도메인 0 · POST 0
콘솔	세 엔진에서 수집	에러 0 · 경고 0

명세 §4-1이 "*파싱이 안 되는 값이 들어와도 프로그램이 죽지 않고 **확인 필요** 상태로 떨어지게만 처리
해 주세요"라고 요구한다. 위의 **받은 적 없는 데이터** 줄이 그 조항을 창업팀이 제공하지 않은 입력으로
확인한 결과다.

교차 브라우저는 playwright 로 세 엔진을 띄워 확인했다. 그 스크립트는 package.json 에 없는 개발 의
존성을 쓰므로 저장소에 포함하지 않았다.

지원 범위

지원한다

- CSV 입력(UTF-8, BOM 유무 무관), RFC 4180 따옴표·이스케이프 처리
- **화면에서 직접 등록(수기)** — 파일 인입과 같은 파이프라인을 탄다
- **PDF 공문에서 표 읽기** — 텍스트 PDF 한정. 좌표로 표를 복원해 후보를 만든다
- 9개 필드 구조화와 원본 파일명·행 번호(PDF는 쪽 번호) 연결
- 정규화 품목명 후보 생성(사전 → 규칙 → 데이터 부족)
- 예외 4종 탐지와 사유·근거 원본 값 제공
- **규격이 줄었을 때 kg당 실질 인상을 환산** — 산출값이며 판정을 바꾸지 않는다
- 상태 5종 결정, 중복 되돌리기, 승인 차단 조건
- **Chromium · Firefox · WebKit(Safari 엔진)** — 세 엔진에서 배포본으로 확인했다

지원하지 않는다 — 만들 수 있었지만 이번에 안 한 것 (미지원 형식)

- **XLSX 입력** — 요건이 "XLSX 또는 CSV 중 1개"라 CSV를 택했다
- **스캔 이미지 PDF** — 글자가 없는 스캔본은 OCR이 필요하다. 그런 파일을 넣으면

읽을 수 없다고 이유를 알려 준다(위 PDF 공문에서 읽기 → 안 되는 것 참조)

- **EML 입력** — 명세가 메일 계정 OAuth를 금지해 착수하지 않았다
- **제공 20건을 벗어난 판정 정확도** — 규격·단위 포맷이 20건 범위를 넘으면 값을 추측하지 않고

확인 필요 로 떨어뜨린다. 죽지 않는다는 것은 따로 확인했다(검증 범위 참조)

범위 밖 — 애초에 만들지 않은 것 (OUT of scope)

명세 개요가 *해커톤 팀은 그 기능을 복제하지 않고 앞단 인박스 흐름을 구현합니다*라고 선을 그었다. 아래는 못 한 것이 아니라 **하지 않기로 한 것**이다.

범위 밖	왜
가격 판정 · 비교 리포트 · 협상 · 대안 건적/RFAQ	기존 ComfoziAI에 이미 구현돼 있다. 우리는 그 앞단 만 만든다
기존 ComfoziAI Production · DB · API 연결	명세가 접근을 금지했다. 출력은 파일로만 내보낸다
회원 · 권한 · 영구 감사 로그	요건이 "불필요"로 명시했다. 로그인이 없는 이유다
실제 Production 전송	명세가 금지했다. 내보내기 는 브라우저에서 파일을 만들 뿐이다
화면 구밈새 · 디자인 완성도	요건이 "화려한 UI 불필요 · 업무 흐름 우선"으로 명시했다
실사용자 과업 완주율 측정	창업팀 로드맵의 베타 테스터 모집이 2026-08-21부터라 이번 기간 밖이다

알려진 제약과 실패한 시도

- 정규화 규칙 기반 경로는 20건에서 실제로 쓰이지 않는다. 제공된 20건이 사전 19종으로 전부 커버되기 때문이다(자체 테스트 출력 { dictionary: 20 }). 2026-08-12에 받은 샘플 공문의 신규 품목 10건을 넣어보니 사전에 없는 이름이라 전부 규칙 경로를 탔고({ rule: 10 }), 후보 생성과 예외 판정이 정상 동작했다. 다만 사전이 20건에서 뽑은 19종뿐이라, **새 품목의 정규화 정확도는 여전히 규칙 수준까지만 보장한다.**
- 표준 단위 밖의 단일 단위는 20건으로 검증되지 않았던 경로였다. 20건의 단위는 PK · BOX · PO 와 복수 병기 KG/단 뿐이라 "표준 집합 밖의 단일 단위" 분기가 한 번도 실행되지 않았다. 샘플 공문에 봉 · 망 · 말 · 팩 · 캔 · 박스 가 나와서 docs/단위검증_테스트.csv 로 이 경로를 따로 검사하도록 추가했다(자체 테스트 8번).
- 단위 표기를 자동으로 보정하지 않는다. 박스 는 BOX 의 한글 표기이고 pk 는 PK 의 소문자지만, 둘 다 표준 집합 밖으로 보아 unit_mismatch 로 분류한다. 1봉 이 몇 PK 인지는 공급사만 아는 정보이고, 명세가 자동 환산을 금지하기 때문이다. 실무에서 쓰려면 한글 단위 매핑 사전이 필요하며, 이는 구현하지 않았다.
- 관찰값은 완전한 원문이 아니라 앞뒤 공백을 다듬은 값이다. CSV 파서가 셀을 읽는 시점에 trim() 을 적용하므로(parseCsv.ts), " PK " 는 observed 에 "PK" 로 들어간다. 화면의 "파일에서 읽은 원본" 문구는 이 범위에서 읽어야 정확하다. 셀 안쪽 공백과 그 밖의 모든 문자는 손대지 않는다. 앞뒤 공백까지 근거로 삼아야 하는 검수는 지원하지 않는다.
- **DOC-020의 안내 문구(1단 = 몇 kg인지 공급사 확인이 필요합니다)는 규격의 기존 / 변경 값에서 단위 토큰을 잘라 만든다.** 숫자를 떼 나머지를 단위로 보는 단순한 방식이라, 단위 표기가 20건과 많이 다르면 문구가 어색해질 수 있다. 이 경우 일반 문구로 대체된다.
- 자체 테스트 초안이 틀렸던 사례 — 하드코딩 검증에서 문서ID를 역순으로 매겼더니 중복 그룹의

기준 항목이 바뀌어 FAIL이 났다. 확인해보니 "문서ID 오름차순 첫 건이 기준"이라는 명세대로 동작한 것이었고, 구현이 아니라 테스트의 가정이 틀린 것이었다. 테스트를 (a) 순서 유지 (b) 순서 반전 두 갈래로 나눠 각각 다른 것을 검증하도록 고쳤다.

- **파일 읽기에 실패해도 이미 검수 중이던 목록은 지우지 않는다.** 처음 구현은 실패 시 상태를

통째로 비웠는데, 담당자가 잘못된 파일 하나를 고른 순간 진행하던 검수가 날아가는 동작이라 오류 배너만 띄우고 목록은 유지하도록 고쳤다. 재검증 과정에서 발견한 결함이다.

- **빈 목록이 localStorage 백업을 지우던 결함** — 저장 로직이 "목록이 비면 백업 삭제"였는데,

화면이 어떤 이유로든 빈 상태로 다시 뜨면 그 순간 백업이 지워졌다. 실제로 개발 중 화면이 한 번 죽었을 때 검수하던 10건이 통째로 사라졌다. 삭제는 사람이 비우기를 눌렀을 때만 일어나도록 고쳤다.

- **검수 메모가 저장되지 않던 결함** — 재업로드 대응으로 항목 키를 uid로 바꿀 때 메모 핸들러만

doc_id를 넘기고 있었다. 핸들러 타입의 파라미터 이름이 docId로 남아 있던 것이 원인이라, 이름을 uid로 바꿔 같은 실수가 반복되지 않게 했다.

- 상태는 브라우저 세션에 두고 localStorage로 백업한다. 요건상 영구 감사 로그·회원 권한이 불필요하고, 심사 시 여러 명이 각자 접속해도 서로 간섭하지 않게 하려는 선택이다.

개발 중 찾아 고친 오류

요건 세부설명의 **"알려진 오류 기록"** 항목이다. 개요에 **"미완성·오류·제약은 실패로 데이터 숨기지 않고 기록합니다"**라고 되어 있어, **고친 것도 숨기지 않고 남긴다.**

전부 정적 검증(타입체크·린트·빌드)을 통과한 상태에서 나왔다. **타입체크와 테스트는 내가 상상한 시나리오만 검증한다.** 아래 열한 개는 실제로 눌러 보거나, 안 밟아 본 경로를 찾아 밟아 보거나, 다른 엔진에서 돌려 보고서야 나왔다.

연 제	무엇이 틀렸나	원인	처리
08-11	파일 읽기에 실패하면 검수하던 목록이 통째로 사라졌다	실패 시 목록을 초기화	실패해도 목록 유지. 잘못된 파일이면 이 유만 알린다
08-11	빈 목록이 localStorage 백업을 지웠다. 실제로 검수하던 10건을 잃었다	빈 상태 저장을 삭제로 취급	삭제는 사람이 비우기를 눌렀을 때만
08-11	검수 메모가 저장되지 않았다	항목 키를 uid로 바꿀 때 메모 핸들러만 doc_id를 넘김	핸들러 파라미터 이름까지 uid로 바꿔 재발 방지
08-12	승인된 항목의 값을 깨뜨려도 승인이 유지됐다. 깨진 값이 출력으로 나갈 수 있었다	승인 후 재판정을 하지 않음	값이 바뀌면 승인을 자동 해제하고 변경 이력에 남긴다

연 제	무엇이 틀렸나	원인	처리
08-13	20건으로 한 번도 실행되지 않은 코드 경로가 있었다	20건 단위가 PK · BOX · PO · KG/단 뿐이라 "표준 집합 밖 단일 단위" 분기가 미실행	docs/단위 검증_테스트.csv 로 자체 테스트 8번에 고정
08-14	검수 버튼 클릭이 통째로 증발했다. 값·메모를 고치고 바로 승인을 누르면 두 번 눌러야 했다	disabled 가 아니라 click 이벤트 자체가 안 생겼다 — mousedown → blur → 리렌더로 mousedown과 mouseup을 받은 요소가 같았다	mousedown 기본동작을 막고 커밋을 click 안에서 일으킨다. 승인·반려·재검토·중복되돌리기 네 버튼 모두. 결과지로 키보드만으로 승인이 가능해졌다 (disabled 는 Tab 포커스를 못 받는다)
08-14	PDF로 넣은 항목의 출처를 되짚을 수 없었다. 목록에 수기로 쓰고 내보내기도 input_method: manual	PDF 경로에 별도 출처 값이 없었다	input_method 에 pdf 추가, source_ref.page_no 신설
08-15	로컬 npm run build 가 실패했다. 앱은 Compiled successfully 인데 게이트가 빨간불	tsconfig 의 include 가 커밋하지 않는 scripts/*-tmp.ts 까지 잡았다	그 패턴만 exclude . 자체 테스트 verify-20.ts 는 검사 대상으로 남겼다
08-15	1kg → 900g 을 "기준이 바뀌었다"고 거절했다	kg과 g을 다른 단위로 봤다. 같은 물리량인데 표기만 다르다	질량·부피를 차원별로 묶어 환산. 이 오판 때문에 예시 데이터에서 실질 단가 계산 값이 한 번도 보이지 않았다 — 심사위원이 보는 화면이 비어 있었다
08-15	kg당 0 으로 표시됐다	0.01원을 정수로 반올림	100원 미만이면 소수 둘째자리를 살린다. 변화율에는 천단위 콤마
08-15	사파리에서만 PDF를 읽지 못했다	pdf.js getTextContent() 가 내부에서 for await (const v of readableStream) 을 돈다. ReadableStream의 비동기 순회 는 Safari에 없다	streamTextContent() 를 리더로 직접 읽는다. 자세한 경우는 PDF 공문에서 읽기 → 사파리에서만 안 됐다

지금 알고 있으면서 고치지 않은 오류

없다. 다만 확인하지 못한 것이 셋 있고, 그건 오류가 아니라 검증 범위의 한계다.

- 애플이 출하하는 실제 사파리 — 엔진(WebKit 26)에서는 확인했으나 실기기는 보지 못했다
- 실사용자 테스트 — 창업팀 로드맵의 베타 테스터 모집이 2026-08-21부터라 이번 범위 밖이다
- 제공 20건을 벗어난 판정 정확도 — 명세가 평가 대상에서 제외했다. 다만 "*"파싱이 안 되는 값이 들어와도 죽지 않고 확인 필요로 떨어지게** 조항은 따로 확인했다(검증 범위 참조)

재업로드와 누적 인입

파일을 다시 올려도 기존 목록을 덮어쓰지 않고 뒤에 이어 붙인다.

중복 판정의 단위는 파일이 아니라 증빙 항목이고, 비교 범위는 "같은 파일 안"이 아니라 **이미 인입된 전체 항목**이다. 검수 대기·보류뿐 아니라 **승인 완료 항목도 비교 대상**에 포함한다. 공급사가 공문을 재발송하거나 담당자가 실수로 같은 파일을 두 번 올렸을 때, 같은 단가 인상이 두 번 반영되는 사고를 잡는 것이 이 기능의 목적이다.

항목	동작
내부 식별자	<code>uid</code> (인입 순번 + 문서ID)를 문서ID와 별도로 둔다. 문서ID를 유일 키로 쓰면 재업로드에서 저장이 실패한다
기준 항목	시스템 인입 순서 (<code>intake_seq</code>)로 정한다. 먼저 들어온 항목이 기준이고 이후 인입분에 중복 의심을 붙인다
같은 파일 재업로드	내용 해시로 감지해 안내 후 진행 여부를 사람이 선택 한다. 조용히 무시하지도, 조용히 넣지도 않는다
복수 플래그 상태	<code>missing_required</code> 가 포함되면 <code>needs_review</code> , 그 외 조합은 <code>on_hold</code>

20건 파일을 두 번 올렸을 때의 결과는 다음과 같다. 1회차 20건은 판정이 바뀌지 않는다.

2회차 DOC-001~015·017·018	["duplicate_suspected"]	on_hold
2회차 DOC-016	["missing_required", "duplicate_suspected"]	needs_review
2회차 DOC-019	["spec_mismatch", "duplicate_suspected"]	on_hold
2회차 DOC-020	["spec_mismatch", "unit_mismatch", "duplicate_suspected"]	on_hold

한계

- 파일 해시는 내용이 완전히 같을 때만 감지한다. 한 행만 고쳐 재송부된 파일은 해시가 달라 안내가 뜨지 않는다. 다만 중복 탐지 자체는 행 단위로 돌기 때문에 같은 내용의 행은 그대로 잡힌다.
- 인입 이력은 브라우저 세션 범위다. 다른 기기·브라우저에서 올린 항목과는 비교되지 않는다.
- 이 동작은 **창업팀 회신(2026-08-11)의 제품 의도를 구현한 것으로, 채점 정답표(20건 1회 업로드)에는 없는 범위**다. 정답표 기준 판정은 그대로 유지된다.

예시 데이터로 바로 보기

파일을 갖고 있지 않아도 **버튼 하나로 전체 흐름을 확인**할 수 있다. 심사는 배포 링크에 직접 접속해 이루어지는데, 파일이 없으면 빈 화면만 보게 되어 만들어 둔 기능을 확인시킬 방법이 없기 때문이다.

결과를 미리 박아 둔 것이 아니다. 앱에 CSV 문자열을 넣어 두고, 파일을 올렸을 때와 똑같이 파서에 통과시켜 같은 판정 경로를 태운다. 자체 테스트 11번에서 이를 확인한다.

> 1차 심사의 URL은 실제 파이프라인이 동작하는 상태여야 합니다. > 결과 화면이 고정값(하드코딩)이거나 스텝 응답인 경우 심사에서 인정되지 않습니다. > — 창업팀 답변 2026-08-12

예시 데이터(10건)에는 예외 4종과 형식 오류가 모두 들어 있다.

문서	답긴 경우
SMP-004	규격 변경 통보 → spec_mismatch
SMP-005	표준 밖 단위(봉) → unit_mismatch
SMP-006	적용일 공란 → missing_required
SMP-007	SMP-002와 내용 동일 → duplicate_suspected
SMP-008	변경단가가 추후 안내 → 형식 오류

창업팀이 제공한 20건은 넣지 않았다. 해커톤 자료를 공개 URL에 실어 두는 셈이 되기 때문이다(규정 4번). 대신 같은 구조의 가상 데이터를 직접 만들었고, 덕분에 20건이 아닌 데이터에서도 예외 4종이 잡힌다는 것을 함께 보여준다. 예시 데이터 역시 문서ID를 바꿔도 판정이 같은지 테스트로 확인한다.

원본 행 원문

각 항목은 파일에 실제로 적혀 있던 CSV 한 줄을 그대로 들고 있고, 상세 화면에서 펼쳐 볼 수 있다.

```
SMP-004,PDF,바른물산(예시),냉동새우살,기존 1kg / 변경 900g,BOX,42000,42000,2026-09-05
```

관찰값은 앞뒤 공백을 다듬고 따옴표를 쉼 뒤라 원문과 다를 수 있다. 검수 담당자가 "파일에 실제로 뭐라고 적혀 있었는지"를 확인할 수 있어야 근거가 되므로 손대지 않은 줄을 남긴다.

따옴표 안에 줄바꿈이 있으면 논리적 한 행이 물리적 여러 줄이 되므로, 파싱이 끝난 뒤에 원문을 다시 자를 수 없다. 파서가 행의 시작과 끝 위치를 기억해 두었다가 함께 돌려준다. 수기 등록 항목은 파일의 행이 아니므로 이 값이 비어 있고, 화면에도 표시하지 않는다.

raw_line 은 내보내기 스키마에 넣지 않았다. 검수 근거이지 앞단이 받아 쓸 데이터가 아니다.

PDF 공문에서 읽기 (추가 요건)

명세의 추가(선택) 요건이다. 필수 7개를 마친 뒤 착수했다.

> 원본 문서 입력을 한 가지 방식으로 추가하고 후보 생성 가능성을 탐색 > · OCR·AI 정확도 목표 없음 · 실패 사례와 미지원 형식 기록

OCR이 필요하지 않았다

받은 샘플 공문 4종을 열어 보니 이미지가 0개인 텍스트 PDF였다. 글자를 읽는 것은 문제가 아니었고, 진짜 과제는 표 구조 복원이었다. 텍스트만 뽑으면 이렇게 붙어 나온다.

```
품목  규격  단위  기존단가  변경단가  적용일자
토마토살사S/04kg/PKPK32,00033,6002026-08-01
```

pdf.js가 글자 조각마다 위치를 함께 주므로, **y좌표로 행을 묶고 x좌표로 열을 갈라** 복원한다.

만드는 동안 걸린 것 세 가지

문제	원인과 처리
머리글 품 목 을 못 읽음	자간을 넓은 제목이라 품 · 목 두 조각으로 들어온다. 공백 조각의 폭 으로 자간과 열 경계를 갈라 합쳤다(자간 11pt 안팎, 열 사이 37pt 이상으로 뚜렷이 갈렸다)
33,600 과 2026-08-01 이 한 칸으로 뭉침	값끼리도 합치려 해서 생긴 문제다. 합치기는 머리글 줄에서만 하고 값은 좌표로 가른다
짧은 품목이 번호 열로 밀림	양 파 중 (가운데 101)이 품목명 (143)보다 번호 (61)에 가까웠다. 칸 수가 머리글 수와 같으면 왼쪽부터 순서대로 짝지어 해결했다. 수가 어긋날 때만 좌표로 맞춘다

사파리에서만 안 됐다 — `getTextContent()` 를 쓰지 않는 이유

Firefox·WebKit을 내려받아 세 엔진에서 돌려 보니 **WebKit만** 실패했다.

```
PDF를 열지 못했습니다. undefined is not a function
```

콘솔 에러도 네트워크 실패도 없었다. 워커 문제인가 빌드 문제인가로 **두 번 잘못 짚었고**, 스택을 떠서야 자리가 나왔다 — pdf.js의 `page.getTextContent()` 안이었다. 그 편의 메서드는 내부에서 이렇게 돈다.

```
for await (const value of readableStream) { ... }
```

ReadableStream의 비동기 순회는 Safari에 없다. 그래서 사파리에서만 터진다. `streamTextContent()` 를 리더로 직접 읽도록 바꿨다(`readTextItems`). 결과는 같고 세 엔진에서 모두 돈다.

pdf.js 빌드도 브라우저·Node가 `legacy` 하나를 쓰게 합쳤다. **이건 이 버그의 원인이 아니었다** (바꿔도 그대로 실패했다). 경로가 하나면 "브라우저에서만 되는 버그"가 생길 자리가 줄어서 남겼다.

배포본에서 다시 재서 확인한 값이다.

엔진	예시 데이터	실질 단가	제공 20건	PDF	콘솔
Chromium 141	10건	+11.1%	확인 대상 4건	1쪽에서 6건	0
Firefox 142	10건	+11.1%	확인 대상 4건	1쪽에서 6건	0
WebKit 26	10건	+11.1%	확인 대상 4건	1쪽에서 6건	0

WebKit은 사파리 엔진이지만 애플이 출하하는 사파리와 완전히 같지는 않다. **실기기 사파리는 확인하지 못했다.**

되는 것

샘플 공문 4종에서 **품목을 하나도 빠뜨리지 않고 읽는다.** 자체 테스트 12번이 이를 고정한다.

공문	표의 품목	읽은 수
다품목 공문(6품목)	6	6

공문	표의 품목	읽은 수
일부 가격 누락 공문(4품목)	4	4 — 추후 안내 까지 그대로
가온푸드 단가변경 공문	3	3 — 적용일 포함
바다원 재발송 공문	1	1

읽은 값은 바로 쓸 수 있는 형태로 다듬는다. 32,000원 → 32000 처럼 콤마와 원 을 떼어, 우리 형식 검증이 읽을 수 있게 한다.

안 되는 것

- 스캔 이미지 PDF는 읽지 못한다. 글자가 없으면 그 사실을 알려 준다. OCR은 지원하지 않는다.
- 문서ID와 공급사는 채우지 않는다. 표 밖(머리말·직인 옆)에 흩어져 있어 위치가 일정하지 않다.

잘못 추측해 넣는 대신 비워 두고 화면에서 받는다.

- 적용일이 표에 없는 공문이 있다. 다품목 공문은 적용일이 표가 아니라 본문의 시행일 에 있어 비워 둔다. 등록하면 필수값 누락으로 잡히고 담당자가 채운다.

- 표 머리글이 아는 이름과 다르거나, 한 칸이 여러 줄로 접히는 표는 읽지 못할 수 있다.

읽은 뒤에도 사람이 판단한다

추출 결과를 바로 목록에 넣지 않는다. 표로 보여 주고 값을 고칠 수 있게 한 뒤, 고른 것만 파일·수기 등록과 같은 파이프라인에 태운다. 읽어 왔다고 판정을 건너뛰면 PDF로 들어온 건만 중복 검사가 빠지는 구멍이 생긴다.

실제로 가온푸드 공문을 읽어 넣은 뒤 20건 CSV를 올리면, 같은 내용인 DOC-001-009-010이 중복 의심으로 잡힌다. 그 공문이 20건의 원본이기 때문이다.

출처는 pdf 로 남긴다

파이프라인은 같이 타지만 출처는 수기 등록과 구분한다.

```
"source_ref": { "input_method": "pdf", "file_name": "가온푸드_단가변경공문.pdf", "page_no": 1, "row_no": null }
```

사람이 추출 결과를 확인하고 고친 뒤 넣는다는 점은 수기 등록과 같다. 그래도 manual 이 아닌 이유는, 값의 근거가 사람의 기억이 아니라 원본 공문이기 때문이다. manual 로 뭉뚱그리면 내보낸 데이터에서 어느 공문에서 나온 인상분인지 되짚을 수 없다. CSV 행에 파일명과 행 번호가 남는 것과 같은 이유로, PDF는 파일명과 쪽 번호를 남긴다.

행 번호 자리에 쪽 번호를 쓰는 이유는 단위가 다르기 때문이다. CSV의 row_no 는 파일의 몇 번째 줄이지만, PDF는 줄이 아니라 표의 한 행이라 줄 번호를 붙일 수 없다. 대신 그 표가 있던 쪽을 남긴다. 목록에는 PDF 1쪽 , 상세에는 파일명 · 1쪽 표에서 읽고 사람이 확인함 으로 보이고, 내보내기에도 source_file_name · source_page_no 로 함께 나간다.

수기 등록

파일이 없어도 화면에서 직접 넣을 수 있다. 공급사가 전화나 구두로 알려진 인상분을 담당자가 기록하는 상황이 이 기능의 용도다.

별도 경로를 만들지 않고 파일 인입과 같은 파이프라인을 태운다. 수기 등록만 따로 처리하면 "직접 넣은 건 중복 검사가 안 된다" 같은 구멍이 생기기 때문이다. 정규화 후보 생성, 예외 4종 탐지, 형식 검증, 중복 판정, 상태 결정이 모두 그대로 적용된다.

실제로 파일에 있는 DOC-001과 같은 내용을 수기로 등록하면 이렇게 뜬다.

```
MAN-002 [duplicate_suspected] 기준 DOC-001 on_hold
2번째 인입분 DOC-001과(와) 내용이 모두 같습니다.
같은 인상분이 두 번 반영되지 않도록 확인이 필요합니다.
```

- 출처는 `input_method: "manual"` 로 남고 **파일명·행 번호**는 `null` 이다. 파일의 한 행이 아니므로 비워 두며, 비어 있다는 것 자체가 "사람이 직접 넣었다"는 근거가 된다.

- 폼에서도 같은 검증기를 쓴다. 값을 입력하는 동안 형식 오류를 바로 보여준다.

규칙을 두 벌로 두면 파일 경로와 폼 경로의 판정이 어긋난다.

- 오류가 있어도 등록을 막지 않는다. `확인 필요` 상태로 들어가고 목록에서 고치면 풀린다.

적용일이 아직 미정인 공문을 먼저 기록해 두는 경우가 실제로 있어서다 (창업팀 샘플 공문의 `"*본사 협의 지연으로 적용일자 미정"` 이 그 사례다).

사람 검수 흐름

값이 바뀌면 판정도 다시 돌린다. 비어 있던 적용일을 채우면 `missing_required` 가 풀리고 상태가 `승인 가능` 으로 바뀐다. 정규화 품목명을 고치면 중복 그룹이 다시 계산된다. 중복은 목록 전체를 봐야 하므로 한 항목만 고쳐도 매번 전체를 다시 판정한다(`recomputeItems`).

- 승인** — 막히는 경우가 셋이다. ① 필수값 누락 ② 정수·날짜로 못 읽는 값 ③ 예외를

수용하려는데 검수 메모가 비어 있음. 버튼을 비활성으로 두고 셋 중 무엇인지와 무엇을 하면 열리는지를 버튼 아래에 적는다

- 반려 / 재검토** — 재검토는 상태를 되돌려 예외에 맞는 상태로 다시 계산한다

- 중복 아님으로 되돌리기** — 같은 품목을 같은 날 두 번 발주하는 정상 건이 있으므로,

사람이 되돌리면 중복 플래그를 상태 계산에서 제외한다. 다시 중복으로 볼 수도 있다

- 검수 메모** — 판단 근거나 공급사에 확인할 내용을 적는 자유 입력란

- 변경 이력** — 값 수정·승인·반려·재검토·중복 판단 변경을 시각·필드·이전값·수정값으로 남긴다

자동 병합·자동 삭제·자동 승인·자동 환산은 하지 않는다. 모든 상태 변화는 사람이 누른 결과다.

구현 현황

필수 요건 7개 전부, 추가(선택) 요건 중 PDF 원본 문서 입력까지 완료했다. 선택 요건은 정본에 2행인데 두 번째 행은 내용이 비어 있어(추가 - - -) 대상이 없다.

요건	상태	어디서 확인하나
① 실행 가능한 검증용 산출물	완료	구동 URL · 실행 방법 · 이 문서 전체
② 증빙 데이터 입력 (CSV + 수기)	완료	처리 흐름 · 수기 등록
③ 구조화 후보와 원본 근거	완료	출력 스키마 · 원본 행 원문
④ 검수 인박스 (상태 5종·필터·복귀)	완료	사람 검수 흐름
⑤ 누락·중복·불일치 탐지	완료	판정 규칙 · 자체 테스트 1·3·4번
⑥ 사람 검수와 변경 이력	완료	사람 검수 흐름 · 자체 테스트 6·7번
⑦ ComfoziAI 전달 데이터 (JSON+CSV)	완료	출력 스키마 · 자체 테스트 9번
추가 · 원본 문서 입력 확장 (PDF)	완료	PDF 공문에서 읽기 · 자체 테스트 12번
추가 · 두 번째 행	해당 없음	정본에 내용이 비어 있다

요구되지 않았는데 한 것은 셋이다. **재업로드 누적 인입**(창업팀 회신이 "*"구현하지 않았다고 감점하지 않습니다"*라고 한 항목), **kg당 실질 단가 환산**, 그리고 **명세 정정 문의**다. 마지막 것은 구현이 아니라 개발 과정에서 나온 일로, 우리 실행 출력으로 명세 안의 어긋남을 찾아 문의했고 창업팀이 전체 참가팀에 정정 공지를 냈다(판정 규칙 DOC-020 항목 참조).

남은 것은 스캔 이미지(OCR)와 EML이며 이유는 **지원 범위** 에 적었다. **2026-08-15 이후 코드는 더 고치지 않는다.**

사용하지 않은 것

- 유료 AI 서비스를 사용하지 않는다. 판정은 전부 규칙 기반이다.
- 실고객 자료·전직장 자료·비밀값을 사용하지 않는다. 입력은 창업팀이 제공한 합성 20건뿐이다.
- 기존 ComfoziAI Production·DB·인증정보에 접근하지 않는다.